

Auditoría técnica: descarga de manga/doujinshi de Hitomi.la y conversión a PDF para Nexus

Fecha: 19 de agosto de 2026

Objetivo

Determinar las vías técnicamente viables para que Nexus reciba una URL o identificador de Hitomi.la, obtenga una galería, descargue sus páginas y entregue un PDF por WhatsApp.

Conclusión ejecutiva

No se identificó una API oficial pública de Hitomi.la para terceros. La arquitectura más sólida separa el problema en: resolver la galería, obtener las URLs de las páginas, descargar con límites y convertir las imágenes a PDF. Heliotrope puede servir para metadatos y búsqueda, mientras que un extractor basado en la estructura actual de Hitomi puede encargarse de las páginas.

1. API oficial

No conviene diseñar Nexus alrededor de una supuesta API oficial encontrada en repositorios informales. Heliotrope se presenta como un mirror de metadatos de Hitomi.la, no como una API oficial de Hitomi.

2. Heliotrope

Heliotrope ofrece una API REST para metadatos y SDKs para Python, JavaScript/TypeScript y Rust. Es especialmente atractivo para Nexus porque el bot está construido sobre Node.js. Su papel recomendado es descubrimiento y metadatos, no asumir que resuelve por sí solo todo el pipeline de descarga.

3. Extractor directo

La alternativa independiente es analizar la estructura de la galería y obtener la información de galleryinfo y files[]. Proyectos como JDownloader muestran que este enfoque es viable y que la resolución de hosts puede depender de lógica JavaScript que cambia con el tiempo.

4. Hitomi Downloader

Hitomi Downloader demuestra que se pueden descargar galerías y exportarlas a PDF/CBZ. Sin embargo, no lo usaría como dependencia directa de Nexus si no existe una interfaz CLI/servidor estable y adecuada. Es más útil como referencia técnica.

5. Implementaciones antiguas

Hay scrapers Python históricos, pero los proyectos abandonados tienen alto riesgo de romperse cuando Hitomi modifica sus scripts o infraestructura. Sirven como referencia, no como base del bot.

6. JDownloader como referencia

El plugin de Hitomi de JDownloader contempla URLs de galerías/reader, gallery ID, galleryinfo, files[], hashes y formatos de imagen. Es una referencia útil para diseñar un extractor robusto.

7. Arquitectura recomendada

Hitomi URL → normalizador → galleryId → Heliotrope/metadatos + extractor de páginas → manifest ordenado → cola de descargas → 3-5 workers → normalización con Sharp → PDF → validación de tamaño → envío por Baileys → limpieza.

8. Descarga concurrente

No conviene usar decenas de conexiones simultáneas. Empezaría con 3-5 workers, timeout de 20-30 segundos y hasta 3 reintentos con backoff exponencial. Así se reduce el riesgo de 429, 403, timeouts y consumo excesivo de RAM.

9. Almacenamiento

No se deben mantener todas las imágenes en RAM. Es preferible usar un directorio temporal por galleryId, guardar cada página en disco, construir el PDF y eliminar los temporales al terminar.

10. Conversión a PDF

PDFKit puede crear PDFs a partir de imágenes, pero si la galería entrega WebP o AVIF puede ser conveniente normalizarlas antes. Sharp es una buena pieza para convertir y controlar dimensiones/calidad. img2pdf es otra alternativa, especialmente si se acepta ejecutar un proceso Python/CLI.

11. Orden de páginas

El orden debe estar determinado por un índice o manifest, no por el orden en que terminan las descargas. Las descargas concurrentes pueden finalizar fuera de orden.

12. Archivos grandes

Hay que establecer límites de páginas y tamaño. Si el PDF supera un umbral práctico, se puede dividir en varias partes. Para manga, CBZ es técnicamente muy conveniente, aunque PDF es más universal para compartir.

13. Cache

Se recomienda cachear por galleryId. Si varios usuarios solicitan la misma galería, Nexus no debería descargarla varias veces. Esto reduce considerablemente red, CPU y tiempo.

14. Cola

Un sistema de cola evita que muchos usuarios disparen cientos de descargas simultáneas. Se pueden limitar los jobs activos y mostrar posición o progreso.

15. Estructura de código

Se recomienda aislar el módulo en src/hitomi/ con index.js, resolver.js, metadata.js, downloader.js, pdf.js, cache.js y utils.js, en lugar de mezclar toda la lógica en handler.js.

16. Comando sugerido

#hitomi como comando principal. Opcionalmente #hitomis para búsqueda y #hitomi pdf / cbz para elegir formato.

17. Qué NO hacer

Evitar scraping ingenuo de etiquetas img, URLs de hosts quemadas en código, 100 conexiones simultáneas y depender de scrapers abandonados.

18. Consideraciones legales

Que un contenido sea accesible técnicamente no significa que su descarga o redistribución esté autorizada. Para un bot público conviene respetar derechos de autor, términos aplicables y límites de uso.

Veredicto

Para Nexus: extractor propio basado en la estructura actual + Heliotrope para búsqueda/metadatos + descarga controlada + cache + Sharp + generador PDF + cola. Es la opción más modular y mantenible.

Referencias técnicas

Proyecto	Referencia
Heliotrope	https://github.com/Saebasol/Heliotrope
JDownloader, plugin HitomiLa	https://github.com/mirror/jdownloader/blob/master/src/jd/plugins/decrypter/HitomiLa.java
Hitomi Downloader	https://github.com/lanyeeee/hitomi-downloader
PDFKit	https://pdfkit.org/docs/guide.pdf
img2pdf	https://pypi.org/project/img2pdf/